

ECE 1508: Reinforcement Learning

Chapter 1: Introduction

Ali Bereyhi

`ali.bereyhi@utoronto.ca`

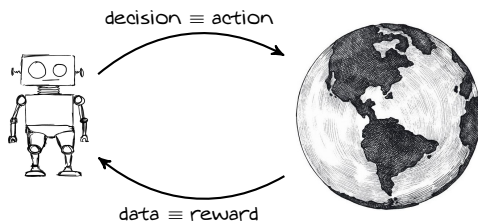
Department of Electrical and Computer Engineering
University of Toronto

Fall 2026

What is Reinforcement Learning?

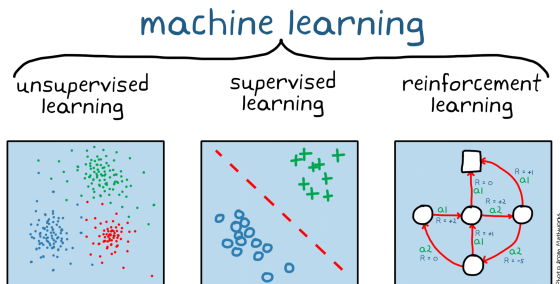
David Silver from *Deep Mind* calls Reinforcement Learning
the art of *decision making*

Of course *David Silver* can say it! But, for us as *beginners* it's better to say
it is all about learning how to interact with environment



We learn many things in our life by this approach

What is Reinforcement Learning?



Reinforcement learning is still a learning problem since

we still learn patterns or behaviors from data $\equiv$ observations

But, it has some fundamental differences to classical learning problems

What is Reinforcement Learning?

Reinforcement learning is *not supervised*

- ↳ *We don't collect data with labels*
- ↳ *We only see some rewards time to time*

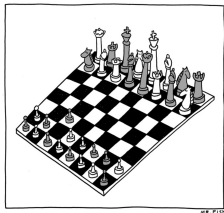
Reinforcement learning does *not* have *conventional dataset*

- ↳ *Each time we decide for an action we get a new feedback*
- ↳ *This feedback contributes to our learning $\equiv$ a new data-point*

In reinforcement learning *time* really *matters*

- ↳ *We should act to see the next data*
- ↳ *It's also different from classical sequential data in the sense that it's *online**

What is Reinforcement Learning?



Best example of a reinforcement setting is a **game**

- In games, we start from **noting**
 - ↳ We know just a bunch of rules
- We **decide** for a move $\equiv$ an action
 - ↳ We then wait for the other side to move
- We decide for next move based on our **observation**

- Reinforcement learning is **not supervised**
 - ↳ We can't label a move as **good** or **bad**!
- Reinforcement learning does **not** have **conventional dataset**
 - ↳ Each move impacts future moves in the game
- In reinforcement learning **time** really **matters**
 - ↳ We can only decide for the next move once we've seen the opponent's move

Reinforcement Learning: Achievements

Reinforcement Learning has been hugely *revisited* in recent years

Alpha-Zero could beat world champions

take a look at *this video* where you could also meet *David Silver* 😊

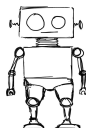
We'll get there at some point, but first let's go back to 1952 and look at Herbert Robbins' *multi-armed bandit* problem which is pretty much

the *most classic* reinforcement learning problem

This helps us develop some intuition

Multi-armed Bandit

There are lots of variants! Let's start with our silly one: *you have developed a **programming robot** as your **course project***

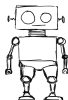


*This **robot** can program in almost all programming languages. You now come up with a **brilliant** idea*

*You plan to **rent** it daily to make **some money**!*

Multi-armed Bandit

Your robot finds two options



Company A



Company B

- Company A which codes in *Java*
- Company B which codes in *Python*

But *none* of these companies has a fixed daily payment

their payments is *randomly* changing every day

Multi-armed Bandit



Company A pays **rent** R_A which is **Gaussian random variable**

- ↳ It has mean $\mu_A = 600 \rightsquigarrow$ it pays in average \$600 per day
- ↳ Its standard deviation is $\sigma_A = 100 \rightsquigarrow$ some days it may even pay \$300



Company B pays **rent** R_B which is **Gaussian random variable**

- ↳ It has mean $\mu_B = 400 \rightsquigarrow$ it pays in average \$400 per day
- ↳ Its standard deviation is $\sigma_B = 200 \rightsquigarrow$ some days it may even pay \$1000

Multi-armed Bandit: *Known Distributions*

Obviously, the money we make depends on our *renting policy*

Policy

$\pi(t)$ is the renting policy which *specifies the selected company on day t*

$$\pi(t) = \begin{cases} A & \text{if we select Company A on day } t \\ B & \text{if we select Company B on day } t \end{cases}$$

Now, the main question is

What is the *optimal* renting policy?

To answer this question we need to *define* what we mean by *optimal*

Multi-armed Bandit: Goal

- + Companies have **stochastic** payments! How can we define **optimality**?
- Well, let's try looking at **expected** return per day

Average Return

Say we rent out for a period of T days, the **average return** is

$$G_T = \mathbb{E} \left\{ \frac{1}{T} \sum_{t=1}^T R_t \right\}$$

where R_t is the rent paid on day t

We now set our goal to **maximize** the **average return**

optimal policy $\equiv$ **maximum average return**

Multi-armed Bandit: Known Distributions

- + But, isn't the answer *obvious*?!
- If we *know* the distributions *yes*!

With *known* distributions, we could simply write

$$G_T = \mathbb{E} \left\{ \frac{1}{T} \sum_{t=1}^T R_t \right\} = \frac{1}{T} \sum_{t=1}^T \mathbb{E} \{ R_t \} = \frac{T_A \mu_A + T_B \mu_B}{T}$$

where T_A and T_B are defined to be

- T_A is the number of days we *rent* to Company A
- T_B is the number of days we *rent* to Company B

We can obviously say that $T_A, T_B \leq T$ and

$$T_A = T - T_B$$

Multi-armed Bandit: *Known Distributions*

The return is

$$G_T = \frac{T_A \mu_A + T_B \mu_B}{T}$$

We can now write

$$\begin{aligned} G_T &= \frac{(T - T_B) \mu_A + T_B \mu_B}{T} = \left(1 - \frac{T_B}{T}\right) \mu_A + \frac{T_B}{T} \mu_B \\ &= \mu_A - \frac{T_B}{T} (\mu_A - \mu_B) \end{aligned}$$

Recall that $\mu_A = 600$ and $\mu_B = 400$, so we have

$$G_T = 600 - 200 \frac{T_B}{T}$$

Multi-armed Bandit: *Known Distributions*

The return is

$$G_T = 600 - 200 \frac{T_B}{T}$$

We know that

$$\text{always rent to A} \iff 0 \leq \frac{T_B}{T} \leq 1 \iff \text{always rent to B}$$

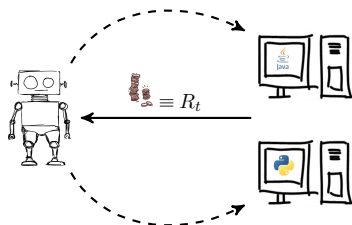
Therefore, *optimal return* is when $T_B/T = 0$

$$G_T^* = 600$$

optimal policy always *rents* to Company A $\iff \pi^*(t) = A$ for all t

- The problem is that we *don't know* the *distributions* in practice!

Multi-armed Bandit



In practice, we can only *observe* payments done each day by the companies
we should *decide* the *renting policy* based on our *observations*

- + OK! It seems *hard* to find an *optimal* policy!
- Well! Let's take a look

Multi-armed Bandit: *Exploring Policy*

We start by a **dumb policy**: we *flip a uniform coin* every day to select company

$$\pi(t) = \begin{cases} A & \text{with Probability 0.5} \\ B & \text{with Probability 0.5} \end{cases}$$

Let's compute the **return** in this case

we denote it as $G_T^{(r)}$ as it's a **random** policy

Multi-armed Bandit: *Exploring Policy*

We have in this case

$$\begin{aligned} G_T^{(r)} &= \frac{1}{T} \sum_{t=1}^T \mathbb{E} \{R_t\} \\ &= \frac{1}{T} \sum_{t=1}^T 0.5 \mathbb{E}_{R_t} \{R_t | A\} + 0.5 \mathbb{E}_{R_t} \{R_t | B\} \\ &= 0.5\mu_A + 0.5\mu_B = 500 \end{aligned}$$

Now comparing to the *maximal return* we could get, we achieve

$$\text{regret} \leftarrow \rho^{(r)} = G_T^* - G_T^{(r)} = 600 - 500 = 100$$

less return!

Multi-armed Bandit: Exploring-Exploiting Policy

- + But, can we get closer to the **maximal return**?
- Sure! We should try to **learn the behavior of the companies**

Let's make another policy: we try each company **once**, and stick to the one who pays higher, i.e., $\pi(1) = A$, $\pi(2) = B$, and

$$\pi(t) = \begin{cases} A & \text{if } R_1 > R_2 \\ B & \text{if } R_1 \leq R_2 \end{cases}$$

Let's compute the **return** in this case

we denote it as $G_T^{(1)}$ as we explore only **once**

Multi-armed Bandit: Exploring-Exploiting Policy

Let's start with finding the return

$$\begin{aligned}
 G_T^{(1)} &= \frac{1}{T} \sum_{t=1}^T \mathbb{E} \{R_t\} \\
 &= \frac{1}{T} \left(\mathbb{E} \{R_1\} + \mathbb{E} \{R_2\} + \sum_{t=3}^T \mathbb{E}_{\pi(t)} \{ \mathbb{E}_{R_t} \{R_t | \pi(t)\} \} \right) \\
 &= \frac{1}{T} \left(\mu_A + \mu_B + \sum_{t=3}^T \Pr \{ \pi(t) = A \} \mu_A + (1 - \Pr \{ \pi(t) = A \}) \mu_B \right)
 \end{aligned}$$

Let's define $\Pr \{ \pi(t) = A \} = p_1$; then, we can write

$$G_T^{(1)} = \frac{1}{T} (\mu_A + \mu_B + (T-2) [p_1 \mu_A + (1-p_1) \mu_B])$$

Multi-armed Bandit: *Exploring-Exploiting Policy*

We have in this case

$$p_1 = \Pr \{ \pi(t) = A \} = \Pr \{ R_1 > R_2 \} = \Pr \{ R_1 - R_2 > 0 \}$$

Let's look at the random variable $\Delta = R_1 - R_2$: R_1 and R_2 are Gaussian

↳ Δ is also a Gaussian variable

↳ Its mean is

$$\mu_{\Delta} = \mathbb{E} \{ R_1 - R_2 \} = \mu_A - \mu_B = 200$$

↳ Its standard variation is

$$\sigma_{\Delta} = \sqrt{\sigma_A^2 + \sigma_B^2} = \sqrt{50000} = 223.6$$

Well, p_1 is readily computed

$$p_1 = \Pr \{ \Delta > 0 \} = Q \left(-\frac{200}{223.6} \right) = Q(-0.89) \approx 0.81$$

Multi-armed Bandit: Exploring-Exploiting Policy

The *average return* is hence given by

$$\begin{aligned} G_T^{(1)} &= \frac{\mu_A + \mu_B}{T} + \left(1 - \frac{2}{T}\right) (0.81\mu_A + 0.19\mu_B) \\ &= (0.81\mu_A + 0.19\mu_B) - \frac{0.62}{T} (\mu_A - \mu_B) \\ &= 562 - \frac{124}{T} \end{aligned}$$

Comparing to *maximal average return*, we have

$$\rho^{(1)} = G_T^* - G_T^{(1)} = 38 + \frac{124}{T}$$

which can be *much less* than $\rho^{(r)} = 100$ if T is large enough!

Multi-armed Bandit: *Exploring-Exploiting Policy*

There is however an obvious problem with this approach

Though better in average, it's **not** that **reliable**!

Reliability Issue

This would not be reliable in general: we could get for one random sample $R_1 \leq R_2$ even though we have $\mu_A > \mu_B$!

- + *Do you think we can make it more reliable?*
- Sure! Let's **explore** a bit more

Multi-armed Bandit: *Exploring-Exploiting Policy*

Let's make the policy more reliable: we try each company *d days* and stick to the one with higher sum payments, i.e., we set $\pi(t) = A$ for $t = 1, \dots, d$ and then compute a *parameter*

$$S_A = \sum_{t=1}^d R_t$$

We then set $\pi(t) = B$ for $t = d + 1, \dots, 2d$ and compute

$$S_B = \sum_{t=d+1}^{2d} R_t$$

We finally set for $t > 2d$

$$\pi(t) = \begin{cases} A & \text{if } S_A > S_B \\ B & \text{if } S_A \leq S_B \end{cases}$$

Multi-armed Bandit: Exploring-Exploiting Policy

We can follow the same lines of calculation

$$\begin{aligned}
 G_T^{(d)} &= \frac{1}{T} \sum_{t=1}^T \mathbb{E} \{R_t\} \\
 &= \frac{1}{T} \left(\sum_{t=1}^d \mathbb{E} \{R_t\} + \sum_{t=d+1}^{2d} \mathbb{E} \{R_t\} + \sum_{t=2d+1}^T \mathbb{E}_{\pi(t)} \{ \mathbb{E}_{R_t} \{R_t | \pi(t)\} \} \right) \\
 &= \frac{1}{T} \left(d\mu_A + d\mu_B + \sum_{t=2d+1}^T \Pr \{ \pi(t) = A \} \mu_A + (1 - \Pr \{ \pi(t) = A \}) \mu_B \right)
 \end{aligned}$$

We now define $\Pr \{ \pi(t) = A \} = p_d$ for $t > 2d$ and write

$$G_T^{(d)} = \frac{1}{T} (d[\mu_A + \mu_B] + (T - 2d)[p_d\mu_A + (1 - p_d)\mu_B])$$

Multi-armed Bandit: *Exploring-Exploiting Policy*

We have in this case

$$p_d = \Pr \{ \pi(t) = A \} = \Pr \{ S_A > S_B \} = \Pr \{ \Delta > 0 \}$$

where we define

$$\Delta = S_A - S_B \sim \mathcal{N}(\mu_\Delta, \sigma_\Delta)$$

It's easy to show that

$$\mu_\Delta = d(\mu_A - \mu_B) = 200d \quad \sigma_\Delta = \sqrt{\sigma_A^2 + \sigma_B^2} \sqrt{d} = 223.6\sqrt{d}$$

p_d is computed as

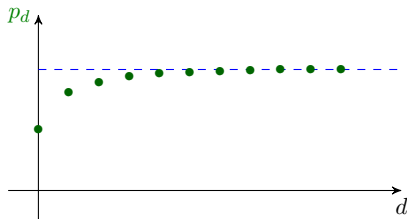
$$p_d = \Pr \{ \Delta > 0 \} = Q \left(-\frac{200\sqrt{d}}{223.6} \right) = Q \left(-0.89\sqrt{d} \right)$$

Multi-armed Bandit: Exploring-Exploiting Policy

The average return is hence given by

$$G_T^{(d)} = p_d \mu_A + (1 - p_d) \mu_B - \frac{2d}{T} (p_d - 0.5) (\mu_A - \mu_B)$$

where our p_d quickly converges to one



So we could say, for $d > 5$ we have

$$G_T^{(d)} \approx \mu_A - \frac{d}{T} (\mu_A - \mu_B)$$

Multi-armed Bandit: *Exploring-Exploiting Policy*

For *slightly large* d , we have

$$G_T^{(d)} \approx 600 - \frac{200d}{T}$$

So, we could *conclude* that

$$\rho^{(d)} \approx G_T^* - G_T^{(d)} = \frac{200d}{T}$$

If we rent out for *some long time*, i.e., $T \gg d$; then, we eventually get to the *optimal return*

$$\lim_{T \rightarrow \infty} \rho^{(d)} = 0$$

Key Task: *Decision Making*

The key task in this problem was *decision making*

- We *train* our robot to *decide* for best employer
 - ↳ This robot is referred to as an *agent*
- Everything would have been easy if we already had *all data available*
 - ↳ Before starting, we had *lots of payment samples*
- We are *observing data* while we are *making decisions*
 - ↳ We find out about payments *after working there*
 - ↳ We are *interacting* with an *environment*

This is a simple example of reinforcement learning

*Let's introduce it and break its *components* down*